

TRƯỜNG ĐẠI HỌC XÂY DỰNG HÀ NỘI  
BỘ MÔN KHOA HỌC MÁY TÍNH

---

## THIẾT KẾ VÀ ĐÁNH GIÁ THUẬT TOÁN

# CHƯƠNG 2

## Đệ quy (*Recursive*)

*Phạm Hồng Phong*

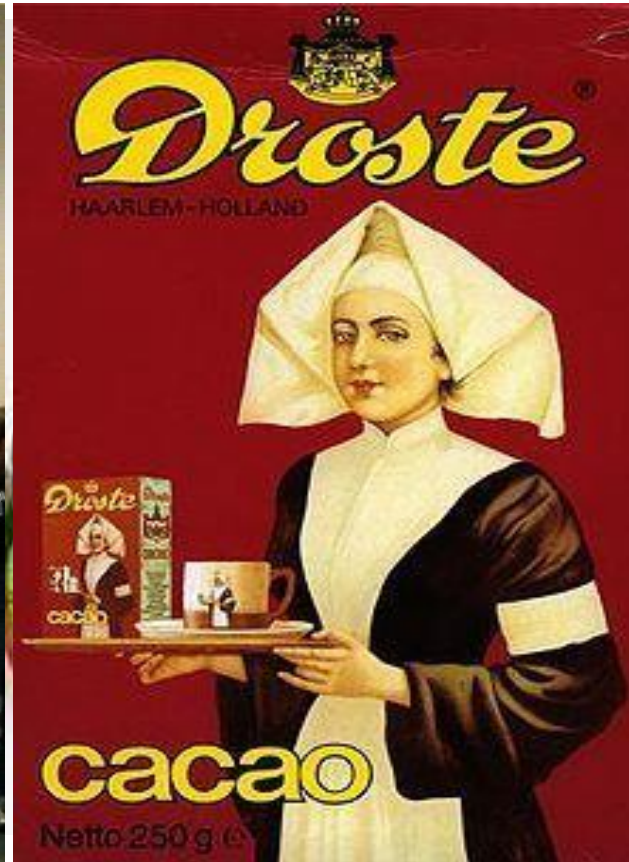
*phongph@huce.edu.vn*

# Nội dung

- 1. Khái niệm**
- 2. Lược đồ giải thuật đệ quy**
- 3. Đánh giá độ phức tạp giải thuật đệ quy**

# § 1. Khái niệm

- Hình ảnh đệ quy



# § 1. Khái niệm

- **Giải thuật đệ quy:**
  - Giải thuật đệ quy: Giải bài toán T được thực hiện cách bằng giải bài toán T' có dạng giống như T.
- **Yêu cầu để giải thuật đệ quy thoả mãn tính dừng:**
  - T' phải “đơn giản hơn” T
  - T' giải được (không cần đệ quy) trong một số trường hợp nào đó (trường hợp suy biến).
- **Ví dụ:**
  - Tính giá trị  $S(n) = n!$   
Ví dụ:  $S(n) = n! \leftarrow n.S(n-1) \leftarrow (n-1).S(n-2) \leftarrow \dots \leftarrow 2.S(1)$

# § 1. Khái niệm

- **Đặc trưng của các bài toán có thể giải bằng đệ quy**
  - Các bài toán **phụ thuộc tham số**;
  - Ứng với các giá trị đặc biệt nào đó của tham số thì bài toán có giải thuật để giải (**trường hợp suy biến**)
  - Trong **trường hợp tổng quát** bài toán có thể quy về dạng tương tự với một bộ giá trị mới của tham số và sau một số hữu hạn lần thì có thể dẫn tới trường hợp suy biến.

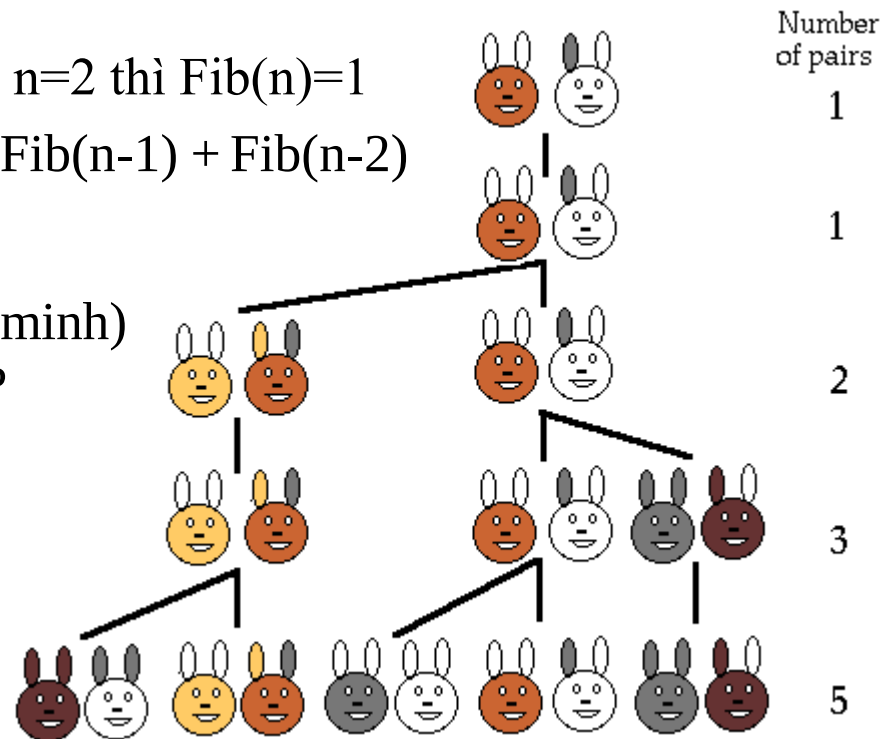
# § 1. Khái niệm

- **Một số ví dụ:**

- Xác định số thứ  $n$  của dãy số Fibonacci 1, 1, 2, 3, 5, 8, 13 ...

- Giải thuật tính  $\text{Fib}(n)$
- TH suy biến  $n=1$  hoặc  $n=2$  thì  $\text{Fib}(n)=1$
- TH tổng quát  $\text{Fib}(n) = \text{Fib}(n-1) + \text{Fib}(n-2)$

- Câu hỏi: Công thức (tường minh) xác định số Fibonacci thứ  $n$ ?



# § 1. Khái niệm

- **Một số ví dụ:**

- Bài toán tháp Hà nội: Chuyển  $N$  tầng tháp từ  $A$  sang  $B$  lấy  $C$  làm trung gian.

Điều kiện: Chỉ có 3 cột để di chuyển; mỗi lần chuyển 1 tầng; trong quá trình chuyển mỗi tầng luôn được đặt trên tầng lớn hơn.

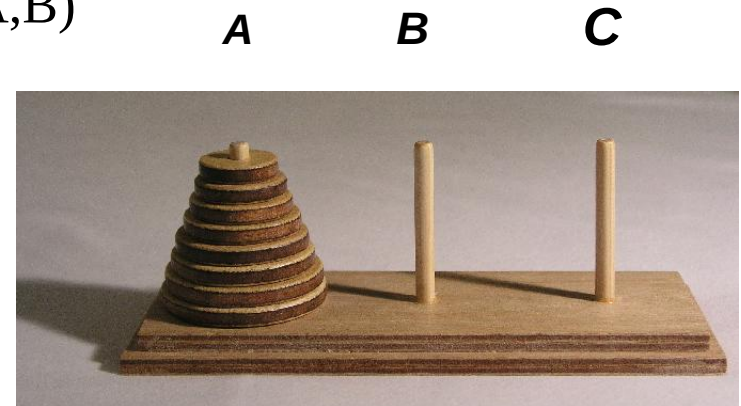
- Giải thuật chuyển tháp CHUYEN( $N, A, B, C$ )
- TH suy biến  $N=1$  thì Chuyen1Tang( $A, B$ )
- TH tổng quát ( $N>1$ )

CHUYEN( $N-1, A, C, B$ );

Chuyen1Tang( $A, B$ );

CHUYEN( $N-1, C, B, A$ );

Trong đó: Chuyen1Tang( $X, Y$ ) là thao tác chuyển một tầng tháp từ  $X \rightarrow Y$



## § 2. Lược đồ giải thuật đệ quy

- **Lược đồ**

```
Recursive_Algorithm( $p_n$ )  $\equiv$   
  if ( $p_n = p_0$ )    //TH suy biến  
    <Thực hiện giải thuật trường hợp suy biến>;  
  else    //TH tổng quát  
    <Lệnh;>  
    Recursive_Algorithm( $p_{n-1}$ );  
    <Lệnh;>  
  endif  
End.
```

$$p_n \leftarrow p_{n-1} \leftarrow p_{n-2} \leftarrow \dots \leftarrow p_0$$



## § 2. Lược đồ giải thuật đệ quy

- Ví dụ

- Tính  $n!$

```
S(n) : int ≡      //Hàm đệ quy tính giá trị  $n$  giai thừa
    if (n==1)
        S = 1;
    else
        S = n * S(n-1)
End.
```

## § 2. Lược đồ giải thuật đệ quy

- Ví dụ

- Tính Fib(n)

```
Fib(n):int ≡ //Hàm đệ quy tính giá trị số Fib thứ n  
    if (n==1 || n==2)  
        Fib = 1;  
    else  
        Fib = Fib(n-1)+Fib(n-2);  
    End.
```

## § 2. Lược đồ giải thuật đệ quy

- Ví dụ

- Tháp Hà Nội

ChuyenThap (n, A, B, C)  $\equiv$  *//Thủ tục đệ quy chuyển tháp n tầng từ A sang B*

**if** (n==1)

Chuyen1Tang (A, B) ;

**else{**

ChuyenThap (n-1, A, C, B) ;

Chuyen1Tang (A, B) ;

ChuyenThap (n-1, C, B, A) ;

**}**

**End.**

Chuyen1Tang (A, B)  $\equiv$  *//Thủ tục chuyển 1 tầng tháp từ A sang B*

print (A, "→", B)

**End.**

# § 3. Độ phức tạp giải thuật đệ quy

- $T(n)$ : độ phức tạp của giải thuật đệ quy với bài toán kích thước  $n$
- $O(1)$ : độ phức tạp thuật toán trong trường hợp suy biến khi  $n=n_0$
- $f(n)$ : hàm biến đổi tham số  $n$
- Nếu giải thuật được thực hiện bằng cách thực hiện  $a$  lần bài toán con với tham số  $f(n)$  thì

$$T(n) = a.T(f(n)) + g(n)$$

( $g(n)$  là độ phức tạp các thao tác ngoài lời gọi đệ quy)

=> Công thức truy hồi xác định độ phức tạp giải thuật đệ quy:

$$T(n) = \begin{cases} O(1) & \text{khi } n = n_0 \\ a.T(f(n)) + g(n) & \text{khi } n > n_0 \end{cases}$$

# § 3. Độ phức tạp giải thuật đệ quy

- Ví dụ tìm công thức truy hồi

- Thuật toán S(n)

$$T_s(n) = \begin{cases} O(1) & \text{khi } n = 1 \\ T_s(n-1) + O(1) & \text{khi } n > 1 \end{cases}$$

- Thuật toán Fib(n)

$$T_{Fib}(n) = \begin{cases} O(1) & \text{khi } n = 1, 2 \\ T_{Fib}(n-1) + T_{Fib}(n-2) + O(1) & \text{khi } n > 2 \end{cases}$$

- Thuật toán ChuyenThap(n,A,B,C)

$$T_{ChuyenThap}(n) = \begin{cases} O(1) & \text{khi } n = 1 \\ 2T_{ChuyenThap}(n-1) + O(1) & \text{khi } n > 1 \end{cases}$$

# § 3. Độ phức tạp giải thuật đệ quy

- Giải công thức truy hồi
  - Sử dụng phép thế liên tiếp

$$T_s(n) = \begin{cases} O(1) & \text{khi } n = 1 \\ T_s(n-1) + O(1) & \text{khi } n > 1 \end{cases}$$

$$T_s(n) = T_s(n-1) + O(1)$$

$$= T_s(n-2) + O(2)$$

$$= T_s(n-3) + O(3)$$

$$= \dots$$

$$= T_s(1) + O(n-1)$$

$$= O(1) + O(n-1)$$

$$= O(n)$$

$$\Rightarrow T_s(n) = O(n)$$

# § 3. Độ phức tạp giải thuật đệ quy

- Giải công thức truy hồi
  - Định lí chính (General Theorem): Với  $n$  là lũy thừa của  $b$ , công thức truy hồi

$$T(n) = \begin{cases} O(1) & \text{khi } n = 1 \\ aT(n/b) + O(n^d) & \text{khi } n > 1, \end{cases} \quad \text{với } a \geq 1, b > 1, d \geq 0$$

Có lời giải là

$$T(n) = \begin{cases} O(n^d) & \text{khi } a < b^d \\ O(n^d \log n) & \text{khi } a = b^d \\ O(n^{\log_b a}) & \text{khi } a > b^d \end{cases}$$

Chứng minh: Sử dụng phép thế liên tiếp

# § 3. Độ phức tạp giải thuật đệ quy

Giả sử  $n = b^k$

Ta có

$$\begin{aligned}T(n) &= aT(b^{k-1}) + O(n^d) \\&= a \left[ aT(b^{k-2}) + O\left(\frac{n^d}{b^d}\right) \right] + O(n^d) = a^2T(b^{k-2}) + \left(1 + \frac{a}{b^d}\right) O(n^d) \\&= \dots \\&= a^k + \left[ 1 + \frac{a}{b^d} + \dots + \left(\frac{a}{b^d}\right)^{k-1} \right] O(n^d) \\&= n^{\log_b a} + \left[ 1 + \frac{a}{b^d} + \dots + \left(\frac{a}{b^d}\right)^{k-1} \right] O(n^d)\end{aligned}$$



## § 3. Độ phức tạp giải thuật đệ quy

$$T(n) = n^{\log_b a} + \left[ 1 + \frac{a}{b^d} + \dots + \left( \frac{a}{b^d} \right)^{k-1} \right] O(n^d)$$

- Trường hợp 1:  $a < b^d$

$$T(n) = n^{\log_b a} + \frac{1 - \left( \frac{a}{b^d} \right)^k}{1 - \frac{a}{b^d}} O(n^d) < n^{\log_b a} + \frac{1}{1 - \frac{a}{b^d}} \cdot n^d = O(n^d)$$

## § 3. Độ phức tạp giải thuật đệ quy

$$T(n) = n^{\log_b a} + \left[ 1 + \frac{a}{b^d} + \dots + \left( \frac{a}{b^d} \right)^{k-1} \right] O(n^d)$$

- Trường hợp 2:  $a = b^d$

$$T(n) = n^d + O(n^d \log_b n) = O(n^d \log n)$$

## § 3. Độ phức tạp giải thuật đệ quy

$$T(n) = n^{\log_b a} + \left[ 1 + \frac{a}{b^d} + \dots + \left( \frac{a}{b^d} \right)^{k-1} \right] O(n^d)$$

- Trường hợp 3:  $a > b^d$

$$T(n) = n^{\log_b a} + \frac{\left( \frac{a}{b^d} \right)^k - 1}{\frac{a}{b^d} - 1} O(n^d) < n^{\log_b a} + \frac{\left( \frac{a}{b^d} \right)^{\log_b n}}{\frac{a}{b^d} - 1} \cdot O(n^d)$$

$$= n^{\log_b a} + \frac{n^{\log_b a}}{\frac{a}{b^d} - 1} = n^{\log_b a} \left( 1 + \frac{1}{\frac{a}{b^d} - 1} \right) = O(n^{\log_b a})$$

# § 3. Độ phức tạp giải thuật đệ quy

- Áp công thức truy hồi

- $T(n) = 2T(n/2) + n^2$

- $a = 2, b = 2, d = 2, a < b^d \rightarrow T(n) = O(n^d) = O(n^2)$

- $T(n) = 2T(n/2) + n$

- $a = 2, b = 2, d = 1, a = b^d \rightarrow T(n) = O(n^d \log n) = O(n \log n)$

- $T(n) = 2T(n/2) + 1$

- $a = 2, b = 2, d = 0, a > b^d \rightarrow T(n) = O(n^{\log_b a}) = O(n)$

# § 3. Độ phức tạp giải thuật đệ quy

- Áp công thức truy hồi

- $T(n) = 4T(n/2) + n^3$

- $a = 4, b = 2, d = 3, a < b^d \rightarrow T(n) = O(n^d) = O(n^3)$

- $T(n) = 4T(n/2) + n^2$

- $a = 4, b = 2, d = 1, a = b^d \rightarrow T(n) = O(n^d \log n) = O(n^2 \log n)$

- $T(n) = 4T(n/2) + n$

- $a = 4, b = 2, d = 1, a > b^d \rightarrow T(n) = O(n^{\log_b a}) = O(n^2)$

# Bài tập

- Trình bày giải thuật đệ quy giải bài toán tháp Hà Nội